Week 10 Progress Report: Resilient Distributed Engine for Cryptographic Forensics

Course: Parallel & Distributed Computing · **Instructor:** Dr. Mohammed Alaa Ala'anzy · **Date:** November 2026

Team: Atabek Bakhytzhanov (240107104) · Begzhan Imangali (240107042) · Karzhaubay Bakdaulet (240107105) · Yerkebulan Kenzhegali (240107051) · Madiyar Yerlanuly (240107040)

1. Executive Summary & Completed Milestones (W6–W10)

In accordance with our approved proposal, all baseline milestones from Week 6 through Week 10 have been implemented, verified, and benchmarked in C/C++ (repository: `github.com/MadiyarYerlanuly/project_pdc`). Specifically: **(1) W6–W7:** Implemented the FIPS 180-4 SHA-256 cryptographic engine (`S5`), verified against official NIST test vectors, and defined core data structures (`TaskChunk`, `HeartbeatPacket`). **(2) W8:** Developed the multi-threaded OpenMP + AVX2 SIMD worker engine (`S4`) and extended it with a hardware-agnostic **OpenCL GPU kernel** (`sha256_kernel.cl`). **(3) W9:** Implemented the Dynamic Master-Worker Task Queue (`S1/S2`) using asynchronous pull-based chunk scheduling. **(4) W10:** Conducted full Strong and Weak Scaling baseline benchmarks prior to fault injection.

2. Architectural Enhancement: Mitigating CPU Starvation via OpenCL GPU Offloading

During W8 testing on a 4-core/8-thread laptop (Intel Core i5-1135G7), we observed that saturating all 8 logical CPU threads (100% CPU load) reduced OpenMP parallel efficiency to **55.69%** due to SMT contention and caused OS scheduling jitter that risks delaying asynchronous MPI Heartbeats. To prevent CPU starvation across our heterogeneous student laptops (which lack NVIDIA GPUs and include Intel integrated graphics and Apple Silicon Macs), we introduced two improvements while keeping all syllabus components intact:

• **Anti-Choke OpenMP Guard (CPU Baseline & Batch Prep):** Reserves 2 logical CPU threads exclusively for the OS and MPI asynchronous network daemon, running OpenMP+SIMD on the remaining 6 threads.
• **Universal OpenCL GPU Offloading:** Offloads the compute-bound SHA-256 loop onto *any* vendor's integrated or discrete GPU. On our test laptop's integrated **Intel(R) Iris(R) Xe Graphics (80 Compute Units)**, OpenCL achieved **197.48 MH/s (85.1× speedup over 1 CPU core)** while keeping host CPU utilization at **~2%**.

3. Experimental Baseline Results (Hypotheses H1 & H3, Without Failures)

Compute Mode (H1 Strong Scaling)	Hardware / Threads	Time (2M Hashes)	Throughput	Speedup	Efficiency	Host CPU Load
W6–W7 Serial Baseline	1 CPU Thread	0.8389 s	2.38 MH/s	1.00×	100.00%	12.5%
W8 OpenMP + SIMD	2 CPU Threads	0.4390 s	4.56 MH/s	1.91×	95.54%	25.0%
W8 OpenMP + SIMD	4 CPU Threads	0.2602 s	7.69 MH/s	3.22×	80.60%	50.0%
W8 OpenMP + SIMD (Anti-Choke)	6 CPU Threads (2 free)	0.2131 s	9.38 MH/s	3.94×	65.61%	75.0%
W8 OpenMP + SIMD (Saturated)	8 CPU Threads	0.1883 s	10.62 MH/s	4.46×	55.69%	100.0%
W8 Universal OpenCL GPU	**Intel Iris Xe (80 CUs)**	**0.0101 s (equiv.)**	**197.48 MH/s**	**85.10×**	**GPU Offload**	**~2.0%**

Weak Scaling & Dynamic Load Balancing (Hypothesis H3): Scaling workload proportionally (1,000,000 hashes per worker node across 1 to 4 worker nodes) yielded aggregate throughputs of **11.61 MH/s (1 node, 100%)**, **22.82 MH/s (2 nodes, 98.23%)**, **33.63 MH/s (3 nodes, 96.53%)**, and **44.07 MH/s (4 nodes, 94.88% weak-scaling efficiency)**. Under asymmetric node speeds (simulating heterogeneous laptops from 1.0× to 5.5× relative speed across 24 chunks), the dynamic pull scheduler achieved **96.06% load-balancing efficiency**, exceeding our >90% H3 target.

4. Next Steps (Weeks 11–15: Fault Tolerance & 5-Laptop Chaos Testing)

• **W11 (Module S3 - Rescuer):** Activate 500ms asynchronous `MPI_Isend` heartbeats, periodic chunk checkpointing, automatic dead-node chunk reassignment (Hypothesis H2), and `Rank 1` Shadow Master replication to eliminate Single Point of Failure.
• **W12–W15:** Deploy across all 5 physical laptops over WLAN and conduct live chaos testing (disconnecting nodes mid-execution).